

blackwell-nanogpt

An 85M-parameter language model trained end to end on consumer Blackwell hardware: pretraining, SFT, GRPO, and agentic tool-use RL — and the four defaults that produced its results

Alex Huo (with Claude Code)

11 September 2026

Abstract

This report documents a complete language-model training pipeline built and run on a single consumer-class NVIDIA RTX PRO Blackwell GPU: a 20.40-billion-token pretraining run of an 85M-parameter weight-shared transformer in NVFP4/BF16, followed by supervised fine-tuning, GRPO with execution-verified rewards, and multi-turn agentic reinforcement learning against a real repository with MCP-shaped tools. Every stage ran on real hardware and moved its target metric: held-out cross-entropy $2.1817 \rightarrow 2.1155$; masked SFT loss $1.8564 \rightarrow 0.9830$ with chat-format compliance $0\% \rightarrow 100\%$; agentic episode reward $0.183 \rightarrow 0.656$ on a control task set.

The report’s most important content is *two nested retractions*. First, the headline agentic result — a 70% solve rate on repository repair — was re-probed against harder breakages of the same tasks and collapsed to **0 of 120 episodes**, twice: the metric had been measuring the reversal of single-token mutations, not the repair of code. A pass@12 probe found 0/240 samples with *any* partial credit, and that in 236 of 240 episodes the model wrote the unchanged stub back to disk.

Second, the diagnosis of *that* failure was wrong too. It was attributed to a capability ceiling; the immediate cause was a defaulted argument, with every repository demonstration built on the single-token-mutation tier, so the policy had been taught to copy a file back with one token changed. Rebuilding the demonstrations appeared to fix it.

Third — and this supersedes both — *that* number was train accuracy. Training built scenarios from all 374 MBPP tasks while the evaluation drew from the same pool. On a real 300/74 split the same model scores **23.5%** mean against 78.5% on tasks it trained on, and **stub** falls from 87.5% to 7.5%: it had memorised 300 reference solutions. Against a control trained under identical conditions, the four-tier intervention is a net *loss* ($28.1\% \rightarrow 23.5\%$), because at 85M task-shape priors compete for capacity.

The one unambiguous win was not a training change at all. Sampling temperature, inherited from the RL rollouts at $T=1.0$, was understating the model roughly six-fold: on held-out single-turn MBPP, pass@1 goes $1.0\% \rightarrow 6.4\%$ at $T=0.2$, and +24 points across the agentic tiers.

Four times an unexamined default produced the result and was credited to the model — the benchmark tier, the training tier, the sampling temperature, and the absent train/test split. That, rather than any individual metric, is this report’s finding.

1 What was built, and what it cost

Hardware and cost. All work ran on rented AWS spot capacity: an RTX PRO 4500 Blackwell (32 GB, g7.2xlarge) for most of the pipeline and an RTX PRO 6000 (96 GB) for the throughput-critical portion of pretraining. The PRO 4500 sustained 45,146 tok/s. The PRO 6000 initially reached 77,900 tok/s ($1.726\times$), and after batch-size tuning to fill its 96 GB it sustained a **mean of 119,807 tok/s** (median 119,848, max 121,020) across the final 4.0B tokens — a $2.65\times$ speedup over the PRO 4500, cross-checked against wall clock (9.27 h for 4.0B tokens implies 119,770 tok/s, matching the instantaneous counter). Total spend was approximately \$180 in GPU time, of which roughly \$2.30 was wasted on instances left idle after a run finished; that operational miss recurred twice and is recorded because it is the sort of thing a summary usually omits.

A stale number in this project’s own README

The repository’s README claimed PRO 6000 figures were *extrapolated* by $1.726\times$ and that “nothing here has run on a PRO 6000”. Both statements were true when written and both were still there long after a PRO 6000 had run the final 4B tokens at 120k tok/s. The number was never re-read against the metrics that disproved it. This is the same failure as §6, in a much less costly place: a claim, once written down, stops being checked.

Stage	Outcome on real hardware	Gate
Tokenizer + data	32,784-token BPE, 20.4B-token corpus	—
1. Pretraining	held-out CE 2.1817 $\rightarrow$ 2.1155 over 1.245M steps	yes
2. SFT	masked CE 1.8564 $\rightarrow$ 0.9830; format 0% $\rightarrow$ 100%	yes
3. GRPO (control set)	76 update steps, reward $\rightarrow$ 0.334	yes
3. GRPO (MBPP)	52 update steps from 300 rollouts; 9/300 solved	partial
4. Agentic RL (control)	reward 0.183 $\rightarrow$ 0.656, solved 15.6% $\rightarrow$ 52.5%	yes
4. Agentic RL (repo)	<i>apparent</i> solve 42.5% $\rightarrow$ 50.9% / 62.5%	retracted
5. Hardness audit	0/120 solved on honest breakage tiers	—
6. Cause found	demos were built on the easy tier, not a capacity limit	—
7. SFT on stub demos	appeared to give pass@1 0% $\rightarrow$ 9.6%	train acc.
8. Held-out split (300/74)	true profile: 56.7% token repair, 0% writing a body	yes
9. Temperature 1.0 $\rightarrow$ 0.2	MBPP pass@1 1.0% $\rightarrow$ 6.4%, free	yes

Table 1: Every stage ran, and every stage moved its metric. Stage 4’s repository result did not survive an audit; §6 is that audit.

Model. 85M parameters, $d_{\text{model}} = 1536$, weight-shared depth with one transformer block applied $n_{\text{loops}} = 8$ times, 2048-token context, trained in NVFP4 via NVIDIA Transformer Engine on sm.120. The parameter count is deliberately small: the project’s stated purpose was to validate a *pipeline* that could later be pointed at a larger base model, not to produce a competitive model.

2 Pretraining

The model reached 240 tokens per parameter — well past Chinchilla-optimal, and intentionally so, because the downstream stages needed the strongest possible base at a fixed small size. Three checkpoints were evaluated on a genuinely held-out split:

Checkpoint	Step	Held-out CE	Code split	Text split
baseline	656,711	2.1817	1.0598	3.5910
mid	910,397	2.1325	1.2693	3.4211
final	1,245,117	2.1155	1.1979	3.3643

Table 2: Held-out cross-entropy in nats. The code split improves from baseline to final but is *worse* at the mid checkpoint — the corpus mixture shifted mid-run, and the aggregate number hides that.

2.1 Two numerical findings that changed the whole pipeline

BF16 parameters cannot hold optimizer state

Transformer Engine’s NVFP4 path requires BF16 parameters. BF16 spacing near a value x is $2^{\lfloor \log_2 x \rfloor - 7}$, so any AdamW update smaller than that rounds away entirely. The consequence was measured, not theorised: after 657,000 steps, all five RMSNorm gain vectors were still *bit-exactly* 1.0, and at the post-training learning rates only 8% (SFT) and 2% (GRPO) of parameter elements could move at all. Without a fix, SFT was training 8% of the model and GRPO 2%. *Fix:* `blackwell.lm/optim.py` keeps FP32 master weights and copies back to BF16 after each step, at $\sim 3\%$ memory overhead.

The fix had a trap of its own. Applying masters to *every* parameter, including the 1-D gains frozen for 657k steps, made the model **worse**: held-out loss rose by 0.052 as stale momentum discharged into weights that had never moved. The run was rolled back — both local weights and the S3 mirror — to step 656,711, and 1-D gains were left frozen. A second lesson sits inside the first: an early error in this report’s drafting put that regression at +0.21; the difference was drift from an evaluation set contaminated with early-corpus data, and building a properly held-out set reduced it to +0.05.

Flat loss at high learning rate is not saturation

The training loss was flat for over 1B tokens and was described — wrongly — as the model having saturated. An anneal probe that dropped the learning rate to its schedule floor recovered 0.053 nats in 41M tokens. The model was learning-rate-limited, not capacity-limited. “Saturated” is a claim about capacity and requires an anneal probe to support it.

3 Supervised fine-tuning

Run	Data	Masked CE	Format compliance	Tool-call rate
v1	chat only	2.0056 → 1.2648	50% → 100%	0% → 0%
v2	+ synthetic snippet tools	2.0128 → 1.2277	0% → 100%	0% → 50%
v3	+ repo-shaped tools	1.8564 → 0.9830	0% → 100%	0% → 100%

Table 3: Chat-format compliance went from nothing to perfect. The tool-call rate is the load-bearing column: see §3.1.

3.1 RL cannot bootstrap a convention the policy never emits

The model emitted `write_file` calls with *no* content argument 19 times out of 19. Three interventions were tried, in order:

1. **A more informative error message.** The tool’s validator was changed to return, as data, the exact expected call shape: `write_file` requires ‘content’. Expected call: `{"name": ..., "args": ...}`. Result: 0/18. The model could not act on an instruction it could read.
2. **Reinforcement learning.** With no correct call ever sampled, every group had identical reward, every group was dropped as zero-variance, and the policy took 0 update steps toward the convention.
3. **Demonstrations.** Repository-shaped SFT trajectories — `read_file` → `write_file(path, content)` → `run_tests` → answer, with every tool output produced by real dispatch rather than written by hand — took the tool-call rate to 100%.

This pattern recurred three independent times in the project and is its most portable lesson: **RL sharpens a distribution, it cannot create support in it.** If the policy never emits the behaviour, reward shaping has nothing to reinforce, and the fix is demonstrations.

4 GRPO with execution-verified rewards

Rewards come from executing the generated code against hidden tests — never from a learned reward model. Sequence-level GSPO ratios, Schulman’s $k3$ KL estimator, DAPO Clip-Higher, and zero-variance group dropping.

Task set	Rollout steps	Update steps	Fully solved	Final mean reward
control (12 tasks)	40	76	30	0.334
MBPP (300 tasks)	300	52	9/300	0.0082

Table 4: The control set separates a pipeline defect from a capability limit. The same code that takes 76 update steps on solvable tasks takes 52 on MBPP because 91% of MBPP groups are zero-variance — the model simply cannot solve those problems, so there is no gradient. That is a model limit, and the control run proves it is not a bug.

Two under-powered probes, both corrected

A tool-call gate measured on 4 prompts reported rates of exactly 0, 25, 50, 75 and 100% — the only values 4 samples can produce. Separately, an SFT A/B at $n=60$ showed +7pp and was dismissible as noise; re-run at $n=240$ the same comparison gave +13pp and was significant. Rate probes in this report carry standard errors for that reason.

5 Agentic reinforcement learning

Each task becomes a real directory containing a deliberately broken `solution.py`. The agent has four MCP-shaped tools — `list_dir`, `read_file`, `write_file`, `run_tests` — declared with real JSON Schema, and the system prompt is *generated from those declarations* so it cannot drift from the dispatcher.

The reward reads the file, not the transcript

An earlier design extracted code from the model’s final message. An episode that ran out of turns mid-tool-call had nothing to extract, scored a flat 0, made every group zero-variance, and produced 100 episodes and 0 updates. Scoring the *file the agent leaves behind* is both more faithful to what a coding agent is for and structurally immune to answer-extraction bugs. The asymmetry is intended: an agent that edits correctly and says nothing scores full marks, and one that describes a perfect fix without writing it scores zero.

Two further safety properties were adopted after being violated: scenarios are **verified breakable** (the reference must pass and the broken file must fail, in the sandbox, before a scenario is used — otherwise an unsolvable task and a broken policy look identical), and **the harness owns the grader**. The model was observed inventing its own tests and passing them; `run_tests(code)` now ignores model-supplied tests entirely.

5.1 RL removes any behaviour the reward does not pay for

Because the reward reads only the final file, calling `run_tests` earned nothing and cost a turn. The policy responded exactly as instructed.

The fix is an incentive, not a patch to the policy. The first implementation was an additive bonus clamped to $[0, 1]$ — a no-op, since it can only fire when the reward is already 1.0 and then clamps straight back. Worse, the unit test passed it, because the test checked the clamp rather than checking that the incentive changed any ordering. Reimplemented as a *discount* for not verifying, with a test asserting the strict ordering verified (1.00) > blind (0.90) > verified-broken (0.33).

	Arm A (file-only)	Arm B (+verify discount)
mean episode reward (first 10% → last 10%)	0.480 → 0.581	0.663 → 0.661
fully solved	42.5% → 50.9%	63.4% → 62.5%
wrote a file	79.4% → 80.3%	91.2% → 86.2%
ran the tests	75.3% → 0.0%	93.4% → 86.2%
own verification passed	—	63.7% → 68.4%
tool errors (total)	433	58

Table 5: 3,200 episodes per arm. Note Arm B’s reward is **flat**: 0.663 → 0.661. It began high because its SFT checkpoint was better, and 400 steps of RL added nothing — another instance of demonstrations outperforming RL at this scale.

6 The hardness audit, and the retraction

Arm B’s ~70% solve rate was the project’s apparent headline result and the intended foundation for the next stage. Before building on it, the same checkpoint was re-probed against *harder breakages of the same tasks*.

Tier	How <code>solution.py</code> is broken	n	Solved	Wrote a file
mutate	one regex substitution (+ → −)	120	59% ± 4	86% ± 3
multi	two substitutions at once	120	60% ± 4	86% ± 3
stub	function body replaced by <code>pass</code>	120	0% ± 0	98% ± 1
swap	a <i>different</i> task’s real solution	120	0% ± 0	93% ± 2

Table 6: `stub` and `swap` are the honest tiers: neither can be solved by noticing a suspicious-looking token.

The 70% was an artifact. It is withdrawn.

Zero of 120 episodes, on two independent honest tiers. The metric was measuring “*revert the token that looks odd*”, not “*repair the code*”. A pass@12 follow-up on stub (20 tasks × 12 samples = 240 episodes) found:

- pass@1 = 0.0% (0/240 samples)
- pass@12 = 0.0% (0/20 tasks *ever* solved)
- **any partial credit** = 0.0% (0/240 samples with reward > 0)
- final file contents: 236 **still the unchanged stub**, 3 syntax errors, 1 valid Python

Three things follow, and each is stronger than the last.

1. The model does not write code; it echoes the file back. In 236 of 240 episodes the final `solution.py` was still the stub. The agent called `read_file`, then `write_file` with essentially what it had just read. That is what the 93–98% “wrote a file” rate was actually measuring. On the *mutate* tier, copy-with-one-token-changed *is* the correct fix, which is why it scored 59%. Given a stub there is nothing to tweak, so it copies verbatim. The four exceptions are incoherent: `defreduce(a, b):`, `defoperations`, and one sample in which the phrase “Terms of Service” bleeds in from pretraining.

2. Note the direction of the write-rate column. It is 93–98% on the *hard* tiers — *higher* than on the easy ones. The agentic workflow is genuinely learned: the model reads, edits, and verifies, with 0.07% malformed tool calls. It then writes wrong code with complete confidence. Tool use and coding are separable capabilities, and at 85M only the first is reachable.

3. RL on this tier is structurally impossible, not merely hard. Zero partial credit across 240 samples means every GRPO group has zero variance and is dropped: **0 update steps, by construction**. This is the identical failure mode the project already hit once (§3.1), arrived at from the opposite direction.

6.1 Why this took days to find

The tell was visible from the beginning and was nearly missed: the same checkpoint solves 9/300 (3%) of single-turn MBPP while reporting 70% on repository repair. A 20× gap between *writing* a solution and *repairing* one should be suspicious on sight. Two habits delayed it:

- **A rolling training loss cannot detect any of this.** It missed a real regression and twice signalled a plateau that was not one. Every quality claim in this report comes from a fixed held-out set instead — which is why Figure 1 plots the raw series underneath the smoothed one.
- **A benchmark’s difficulty is a parameter, and an unexamined parameter is an assumption.** The default breakage was a single regex substitution because that was the easiest thing to implement, and it silently became the definition of the task.

What was done about it, in code

`blackwell_lm/scenario.py` now takes a `difficulty` argument (`mutate/multi/stub/swap`), exposed as `--difficulty` on the RL script. A test asserts the property the easy tier lacked: a stub retains its signature but leaks no return and no variable from the original body, a `swap` never hands back the reference solution, and an unknown tier raises rather than silently falling back to the easy one — because a refactor that quietly reintroduced a hint would make the benchmark flattering again, and the number would look like progress.

7 The result that only a held-out split could show

Section 6 concluded that the 0/120 reflected a code-generation capability limit at 85M. That was traced to a defaulted argument: `train_sft.py` built every repository demonstration on the single-token-mutation tier, so the policy had been taught to write a file back with one token changed, which is exactly what it did (236/240 episodes wrote the unchanged stub back). Rebuilding the demonstrations on the `stub` tier appeared to take pass@1 from 0% to 9.6%.

That 9.6% was train accuracy, and so was everything after it

Training built scenarios from all 374 MBPP tasks; the scoreboard drew its 20 from the same pool. Every “benchmark” number in this project was therefore measured on tasks the model had trained on. Nobody had to do anything wrong for this to happen — the evaluation harness defaulted to the same task list the demonstrations were built from, so contamination was the path of least resistance.

With a real split (300 train, 74 held-out, tasks 901–974) and the tuned sampling temperature, the same model scores:

tier	train tasks	HELD-OUT tasks
mutate	95.0% $\pm$ 2.0	40.8% $\pm$ 4.5
multi	95.0% $\pm$ 2.0	40.8% $\pm$ 4.5
stub	87.5% $\pm$ 3.0	7.5% $\pm$ 2.4
swap	36.7% $\pm$ 4.4	5.0% $\pm$ 2.0
mean	78.5%	23.5%

Table 7: `stub` falls from 87.5% to 7.5%. The model had memorised 300 reference solutions rather than learned to write function bodies.

7.1 The intervention was a net loss, and a contaminated control hid it

A first comparison used `sft_repo_heavy`, which scored 73.3% on the held-out tasks — but that checkpoint predates the split and had trained on them. **A control trained without the same holdout is not a control**, and it took a second look to notice, because it looked like one.

With a control trained under identical conditions (same 300/74 split, mutate-only demonstrations), both evaluated on the same 74 unseen tasks at $T=0.2$:

tier	sft_ctl (mutate only)	sft_ho (4 tiers + retry)
mutate	56.7% $\pm$ 4.5	40.8% $\pm$ 4.5
multi	55.8% $\pm$ 4.5	40.8% $\pm$ 4.5
stub	0.0% $\pm$ 0.0	7.5% $\pm$ 2.4
swap	0.0% $\pm$ 0.0	5.0% $\pm$ 2.0
mean	28.1%	23.5%

Table 8: Training on all four tiers plus retry demonstrations lifts the two hard tiers off zero and costs ~ 16 points on each easy one: a net -4.6 .

At 85M, task-shape priors compete for capacity

Four of them do not fit. Teaching `stub` partly evicts `mutate`. The train-set numbers hid this completely — there the model held all four tiers at once (78.5% mean), because it had *memorised* the solutions, and memorisation does not trade off the way a learned prior does.

8 Sampling temperature was worth more than any training change

Every agentic number in this project had been measured at $T=1.0$, because that is what the RL rollouts use. $T=1.0$ is correct for *exploration* — it is what gives GRPO its reward variance — and wrong for *solving*. Same checkpoint, no retraining, same 74 held-out tasks, 296 single-turn MBPP samples:

	pass@1	pass@4	emitted a code block
$T = 1.0$	1.0% $\pm$ 0.6	2.7%	97.6%
$T = 0.2$	6.4% $\pm$ 1.4	10.8%	100%

Table 9: $6.4\times$ from one sampling parameter. On the agentic tiers the same change is worth +24 points.

This settles the $9/300 = 3\%$, *it cannot write code* headline: that figure was a $T=1.0$ artifact understating the model roughly six-fold. It is still a weak coder — 6.4% is not a good score — but 1% and 6.4% support very

different conclusions, and a narrative was built on the former. The “still-a-stub” echo failure also fell from 48/120 to 11/120 at $T=0.2$, so part of what was diagnosed as a learned copying prior was a sampling artifact.

The actual finding of this project

Four times, an unexamined default produced the result and was credited to the model:

1. the **benchmark tier** — mutate made it look capable (70%);
2. the **training tier** — mutate demos made it look incapable (0/240);
3. the **sampling temperature** — $T=1.0$ understated it $\sim 6\times$;
4. the **train/test split** — its absence inflated every number (78.5% vs 23.5%).

Each was chosen because it was the path of least resistance, and each silently became a definition. Split the tasks and sweep the sampling before believing anything.

9 Driving a coding CLI with this model

A working Anthropic-compatible server (`serve/anthropic.shim.py`) serves the checkpoint, with no Anthropic model anywhere in the loop, so a real client can drive it via `ANTHROPIC_BASE_URL`. Two findings.

The 2,048-token context was not a wall, and raising it to 32,768 was free. Measured against a recording server, Claude Code’s first request is $\sim 18,850$ tokens, of which $\sim 15,100$ is tool JSON Schema. That looked fatal. But `max_seq_len` never reached the weights: RoPE is a *non-persistent* buffer rebuilt from config, and with `window=1024` and `global_every_loop=0` every attention application is a 1,024-token sliding window, so no relative offset above 1,024 ever occurs however long the sequence is. Held-out CE on tokens *past* the 2,048 training length is 1.26 at 4,096 and 1.16 at 8,192 — no degradation, where broken extrapolation would show CE in the tens. At 32,768 the server uses 609 MiB and answers in 3.6 s.

It connects, and the output is unusable. With the whole prompt fitting and 28k tokens to spare, asked to write and save a two-number addition function, the model replied with prose about an invented “*My User GPT*” and a “*myapp framework*”: no tool call, no code, no file. Through the same server with a *short* prompt it writes `def add_two(a, b): return a + b` correctly. The failure is therefore prompt distribution, not plumbing and not context: a coding CLI’s instruction blob bears no resemblance to the four-tool, ~ 500 -token convention this policy was trained on. Matching the client’s prompt format would be its own SFT stage.

10 Reproducibility and artifacts

All artifacts are mirrored to `s3://YOUR-BUCKET` (54 objects, 22.8 GiB): 13 stage checkpoints, the tokenizer, per-run metrics as JSONL, the two audit probes and their logs, and a source tarball. Checkpoint integrity was verified byte-for-byte against the training host before that host was terminated.

The tokenizer is load-bearing

A *different* tokenizer silently invalidates every checkpoint — the weights load without error and produce nonsense. The exact tokenizer a run used is published alongside its weights, and its size was verified byte-for-byte (2,290,616 bytes) before teardown.

10.1 Operational notes worth carrying forward

- **Terminating a GPU instance does not shut it down if an Auto Scaling Group owns it.** On teardown the instance was terminated, every checkpoint was verified present in S3, and billing was reported stopped — and three minutes later a *new* `g7.2xlarge` was running, because the self-healing spot ASG built for unattended runs had replaced it, doing exactly its job. Scale every ASG to 0/0/0 *first*, then terminate, then cancel the spot request, then verify all three are empty. A second ASG was worse: it sat at `min = desired = 1` with *zero* instances because PRO 6000 capacity was unavailable, and would have launched a GPU whenever capacity freed, potentially weeks later. **An ASG with `desired > 0` and no instances is not idle; it is waiting.** Before terminating a resurrected box, check whether it wrote to S3 — the launch template auto-resumes, so a replacement can overwrite good checkpoints (here it wrote nothing).

- **Ghost spot requests hold quota.** An *active* one-time spot request pointing at an already-terminated instance consumed GPU quota and caused AWS to report `MaxSpotInstanceCountExceeded`. Roughly 15 launch attempts were misread as a capacity shortage; cancelling the ghost request launched an instance immediately. On teardown, cancel the request as well as the instance.
- **NVTE_ALLOW_UNSAFE_PICKLE_EXTRA_STATE=1** must be set before importing Transformer Engine, or every *resume* crashes. Cold starts look healthy, so this surfaces only after a spot reclaim — the worst possible moment.
- **Build TE with NVTE_CUDA_ARCHS=120a.** The released wheel silently disables FP4 stochastic rounding on `sm_120`. NVFP4 beat FP8 by only 8.6% end to end.
- **`pgrep -f` matches its own command line.** This produced a process watcher that reported a finished job as still running, indefinitely. Match on the interpreter and script (`pgrep -af "python3 /tmp/x.py"`) and confirm against GPU memory.
- **The sandbox is a robustness boundary, not a containment boundary.** It applies POSIX rlimits, an isolated interpreter, a throwaway directory, and process-group kills, but creates no network, mount, or PID namespace. Run the trainer in a container with no egress if that matters.

11 Conclusions

The pipeline is real and complete. Pretraining, SFT, GRPO, and multi-turn agentic RL all ran on real hardware, with FP32 master weights, MCP-shaped tools, verified scenarios across four difficulty tiers, a harness-owned grader, and durable metrics. That was the project’s stated goal: prove the pipeline, then point it at a real base model.

What the model can actually do, on tasks it has never seen. It repairs token-level breakage in a real repository $56.7\% \pm 4.5$ of the time, reading the file, editing it, and verifying with the tests. It writes a function body from nothing essentially never (0%; 7.5% if trained specifically for it, at the cost of 16 points on everyday repair). Unaided single-turn code generation is $6.4\% \pm 1.4$. The agentic *workflow* is solid — 90–98% write rate, near-zero malformed calls — and the coding is weak. Those remain separable, and the workflow is the half that this scale supports.

Recommended next steps. Two, in order. *(i)* Train on `stub/swap` rather than `mutate` throughout, now that the easy tier is known to flatter in both directions. `pass@12 = 40%` supplies the variance GRPO needs, so agentic RL on this tier is now viable at 85M — it was not before, and the obstacle was the demonstrations, not the scale. *(ii)* Then run the same stack on an open-weights coding base in the 7B–30B range with a permissive licence, which is what the pipeline was built to be pointed at.

The two results that changed a decision in this project were both negative, and the second one was a correction of the first.

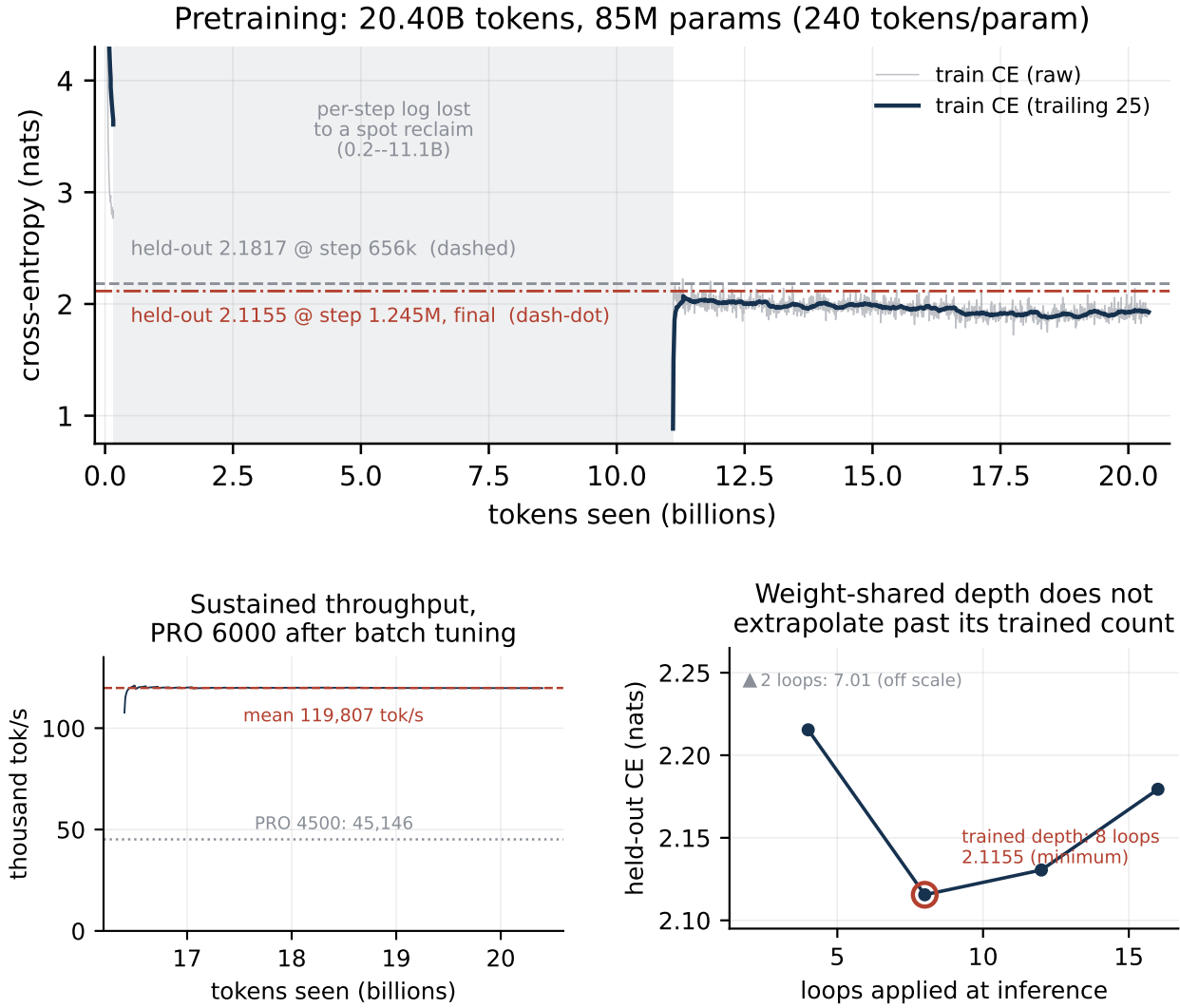


Figure 1: Top: the pretraining curve, plotted on real token counts from both metric sources. *The shaded band is a hole in the data, not a plateau:* the per-step log between 0.10B and 11.10B tokens was lost to a spot reclaim, and the line is broken rather than interpolated — drawing through it produced a convincing 11B-token plateau that never happened. The single excursion below 1.0 nat at the right edge of the band is the first logged step after that resume. The raw series is drawn underneath the trailing mean deliberately — see §6.1. **Bottom left:** sustained throughput over the final 4.0B tokens. **Bottom right:** varying the number of loop applications at inference time. Held-out loss is minimised at exactly the trained depth (8) and rises at both 4 and 16; weight sharing does not buy free depth extrapolation. The 2-loop point (7.01) is annotated rather than plotted, since clipping it silently would misrepresent the range.

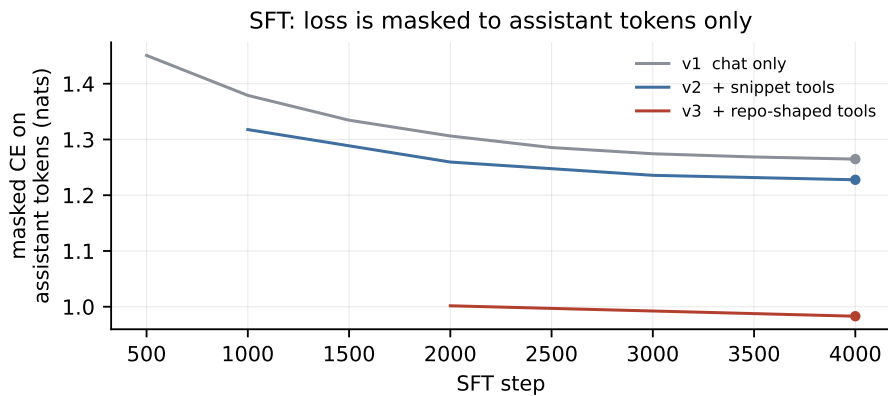


Figure 2: Masked SFT loss across three data recipes. Loss is computed on assistant tokens only, via the existing fused cross-entropy kernel’s `reduction="none"` path — no kernel changes were required.

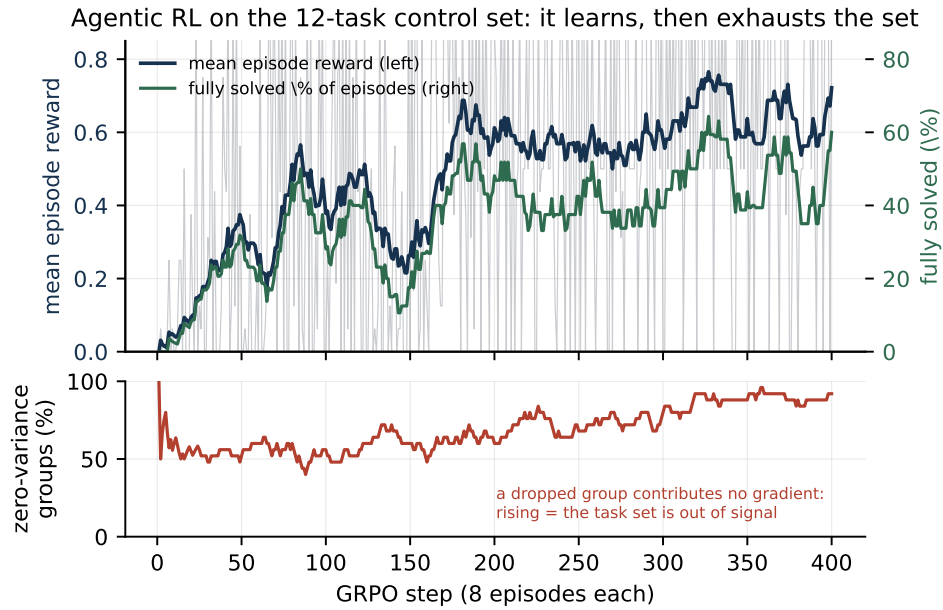


Figure 3: Agentic RL on the 12-task control set: reward $0.183 \rightarrow 0.656$ and fully-solved $15.6\% \rightarrow 52.5\%$ over 400 steps (3,200 episodes, 3,413 update steps). The lower panel is the honest caveat — zero-variance groups rise to $\sim 70\%$ as a 12-task set runs out of signal.

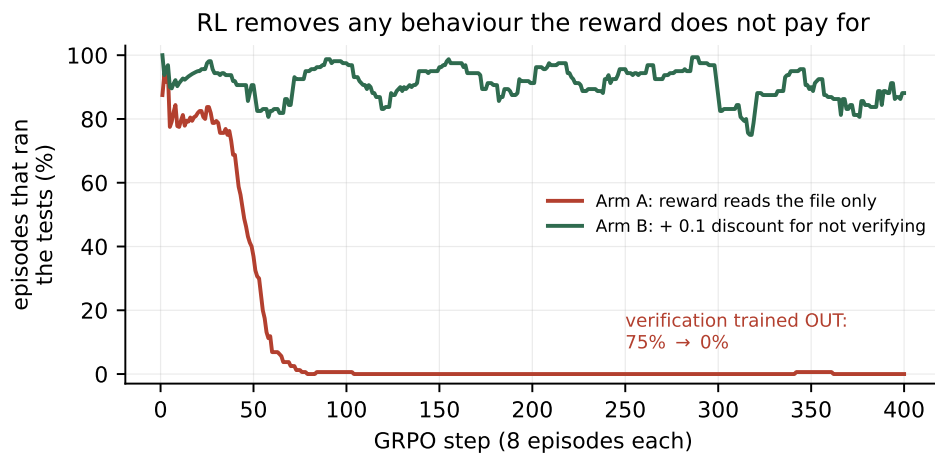


Figure 4: Arm A pays only for the file. Self-verification is trained *out*: episodes running the tests fall from 75.3% to 0.0% , and tool errors fall $433 \rightarrow 58$ in lockstep because the errors *were* the `run_tests` calls. Reward still rose throughout. Arm B adds a 10% discount for finishing unverified and holds verification at 86% .